Delivery semantic (MOM)	Descrizione	Meccanismi in uso	Supporta Non Idempotent
At-least once	non si perdono messaggi, ma potrebbero essere consegnati più volte	Consumer: ack e duplication filtering Producer: timer sul ack per riconsegna	Si, dato che il consumer implementa duplication filtering
At-most once	si potrebbero perdere i messaggi e non si riconsegnerebbero mai	Nessuno	No
Exactly once	messaggio consegnato una volta sola	il MOM implementa duplication filtering su i messaggi del producer	Si, grazie a duplication filtering
Transaction based	I messaggi sono eliminati solo dopo che sono stati letti e processati dal consumer	MOM e consumer si comportano come in presenza di una transazione e rispettano ACID	Si, grazie al ACID
Timeout based	Si assicura che i messaggi siano consegnati almeno una volta prevenendo il doppio consumo	un messaggio appena letto diventa invisibile e se non cancellato torna visibile dopo che finisce il visibility timeout	Si, dato che se il consumer fallisce il processamento, torna a leggere il messaggio originale

Delivery semantic (RPC)	Descrizione	Meccanismi in uso	Supporta Non Idempotent
At-least once	servizio eseguito almeno una volta	Client: request retry Server: nessuno	No, dato che c'è request retry e nessun duplication filtering
At-most once	il servizio è eseguito at-most once	Client: request retry Server: duplication filtering e response retransmission	Si, grazie a duplication filtering e response retransmit
Exactly once	il servizio è eseguito exactly once	Quelli base non bastano, serve anche fault tolerance con write-ahead logging	Si, grazie a duplication filtering, response retransmit e WAL

Sistema MOM	Delivery semantic	Delivery model	Queue type	Replication	Pattern di riferimento	Message retention
Amazon SQS (messaggi)	Timeout based, con visibility timer	Pull: short poll, chiedo a un subset di server. Long poll chiedo a tutti i server	Standard: ordine non garantito e duplicati. FIFO: ordine e nessun duplicato garantiti	Garantita. Nella stessa regione, i messaggi sono su più server	Message queue	I consumer cancellano i messaggi dopo max 14 giorni
Kafka (stream di eventi)	At-most: se acks=0. At-least: se acks=1. Exactly: se acks=all.	Pull	FIFO, con ordine garantito all'interno di una partizione	Si, con replication factor	Publisher-subscriber	I messaggi restano nella coda in base alla dimensione o al retention time
RabbitMQ	At-most (default): autoAck=True. At-least: autoAck=False	il MOM implementa duplication filtering su i messaggi del producer	FIFO con ordine garantito a queue level	Si, viene supportata attraverso i cluster e le quorum queues	Entrambi i pattern sono realizzabili	Dopo la lettura i messaggi vengono cancellati dalle code

Tecnologia	Access	Location	Replication	Failure	Concurrency	Sincronia
Sun RPC	Parziale, serve sapere il nome della procedura e i parametri	Parziale, tramite il port mapper serve solo hostname	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	No, le procedure eseguono in mutua esclusione	No, supporta solo procedure sincrone
Java RMI	Si, attraverso pattern proxy, permette di manipolare oggetti remoti	Parziale, tramite RMI registry che mantiene gli oggetti remoti	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, gli oggetti devono essere invocati in contesto thread-safe	No, supporta solo procedure sincrone
RPyC	Si, con pattern proxy via netrefs, permette di manipolare oggetti remoti	Non è garantita, serve hostname e porta	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, ci sono diverse configurazioni ma vanno gestiti i lock	Si, supporta procedure asincrone
Go RPC (net/rpc)	Parziale, serve sapere il nome della procedura e i parametri	Non è garantita, serve hostname e porta anche se si usa il registrer nel server	No. Ma se introduco un proxy server side posso distribuire il carico	Non è garantita	Parziale, si usano le goroutine ma vanno gestite	Si, supporta procedure asincrone
gRPC	Garantita, tramite Protobuf	Non è garantita, serve hostname e porta	Parziale, tramite un Load balancer esterno	Parziale, permette di fare retry e request hedging se configurato	Parziale, la gestione è application dependent	Si, supporta procedure asincrone

Algoritmo mutua esclusione	Funzionamento	Garanzie, vantaggi e svantaggi
Lamport	Pi invia una richiesta con timestamp ti a tutti gli altri processi e inserisce REQ(ti,Pi) nella sua coda locale. Pj ricevuta la richiesta REQ(ti,Pi), la inserisce nella coda e manda un ACK a Pi. Pi entra in CS se e solo se, REQ(ti,Pi) precede tutte le richieste nella sua coda locale e ha ricevuto gli ACK di tutti gli altri processi con timestamp >ti. Uscito da CS, Pi rimuove REQ(ti,Pi) dalla coda e manda un messaggio di release a tutti gli altri processi. Quando Pj riceve la release rimuovo REQ(ti,Pi) dalla coda	Garantisce: liveness grazie alla release, safety e ordering grazie a timestamp. Svantaggi: serve failure detection per evitare che nessunon non entri in CS, i processi possono essere colli di bottiglia e serve conosce tutti i membri
Agrawala	Pi invia una richiesta con timestamp ti a tutti gli altri processi e inserisce REQ(ti,Pi) nella sua coda locale. Pj ricevuta la richiesta REQ(ti,Pi), risponde con REPLY se non è in CS e non vuole entrarci, se è in CS non risponde e mette la richiesta in coda e se vuole entrare in CS confronta i timestamp e agisce di conseguenza. Pi entra in CS se ha ricevuto gli REPLY di tutti gli altri processi con timestamp >ti. Uscito da CS, Pi rimuove REQ(ti,Pi) dalla coda e manda un messaggio di REPLY . Quando Pj riceve la release rimuovo REQ(ti,Pi) dalla coda	Garantisce: safety,liveness e ordering come Lamport ma costa 2(N-1) e non 3(N-1). È totalmente distribuito, meno messaggi di Lamport. Svantaggi: come Lamport
Token ring	I processi vengono organizzati in un anello dove circola un token in maniera unidirezionale. Il token viagga tra un processo e l'altro e chi possiede il token può entrare in CS se deve oppure passarlo al prossimo processo.	Garantisce: safety e liveness (NS,ND). Svantaggi: si consuma banda per passare il token, il token se perso va rigenerato, un fallimento può portare a multipli token e serve riconfigurare il ring dopo un crash
Maekawa	Un processo entra in CS dopo aver raggiunto il quorum di un sottoinsieme di processi. Tutti i sottoinsiemi hanno la stessa cardinalità K e un processo può votare solo una volta. Un processo P per entrare in CS, manda una richiesta ai membri di Vi e aspetta le reply. Un processo Q in Vi ricevuta la richiesta, se è in CS o ha già risposto salva la richiesta nella sua coda, altrimenti risponde. Ricevute tutte le reply P entra in CS. Una volta uscito manda una release ai membri di Vi. Ricevuta la release Q estrae una richiesta dalla coda e manda una reply.	Garantisce: safety grazie alle intersezioni non nulle dei sotto insiemi. Vantaggi: algoritmo più performante con N>4 $3\sqrt{N}$ Svantaggi: deadlock e quindi liveness non garantita se più processi richiedono di entrare in CS

Soluzione istruzioni sensitive	Descrizione	Meccanismi	Vantaggi e svantaggi
Hardware-assisted virtualization	Hypervisor si affida al HW per intercettare le istruzioni sensitive e privilegiate automaticamente	Introduce la root mode (level 0) per l'hypervisor e non-root mode (level 0) per il guest OS	Vantaggi: risolve ring deprivileging e compression e nessun overhead Svantaggi: serve HW specifico
Fast binary translation	Hypervisor riscrive le istruzioni sensitive al volo	Hypervisor scansiona il codice e sostituisce blocchi di istruzioni privilegiate con blocchi funzionalmente equivalenti	Vantaggi: risolve ring compression. È la prima soluzione ideata. Svantaggi: maggiore complessità e grosso overhead
Paravirtualization	modifica il guest OS per evitare o netraulizzare le istruzioni sensitive non privilegiate	Sostituisce le operazioni non privilegiate con hypercalls, dove l'hypervisor prende il controllo e restituisce il risultato al guest OS	Vantaggi: più pratica rispetto a fullvirtualization su non HW-assisted, overhead più basso rispetto a FBT. Svantaggi: richiede accesso al sorgente del guest OS

Pattern	Scopo	Meccanismo	Vantaggi e svantaggi
Circuit breaker	Prevenire che un guasto di rete o di servizio faccia un effetto cascata su altri servizi	Il client invoca un servizio remoto tramite un proxy che si comporta come un circuito.	
Database per service	quale architettura di database utilizzare	Ogni servizio mantiene i propri dati privati persistenti e accessibili solo tramite API. Le transazioni del servizio coinvolgono solo il DB del servizio	Vantaggi: aumenta il disaccopiamento tra servizi e permette più flessibilità nella scelta del DB. Svantaggi: più complessità di gestione e transazioni tra servizi
SAGA	Mantenere la consistenza nei DB dei servizi senza utilizzare transazioni distribuite	Le cross-transaction vengono realizzate come una sequenza di transazioni locali che aggiornano il DB e pubblicano un evento che innesca la prossima transazione	
CQRS	Come interrogare multipli servizi? Come separare il carico di lavoro di letture e scritture per scalarle indipendentemente?	Si usa un view DB, una replica read-only che supporta le query.	Permette di separare il carico di lavoro delle read da quello delle write

Tipo virtualizzazione	Descrizione	Vantaggi e svantaggi
Virtualizzazione di sistema (tipo 1)	esegue direttamente sul HW e fornisce funzionalità di virtualizzazione integrate in un OS minimo	Maggiori performance rispetto al tipo 2 ma serve modificare il guest OS e conoscere il suo codice sorgente
Virtualizzazione ospitata (tipo 2)	esegue all'interno del OS host ed utilizza i driver e system call del OS host	Sfrutta il sistema operativo host per la gestione dei dispositivi e i servizi di basso livello. Non occorre modificare il guest OS ma ha performance peggiori del tipo 1

Tipi di virtualizzazione leggera	Descrizione	Vantaggi e svantaggi
MicroVM	Piccolo hypervisor specializzato per eseguire macchine virtuali leggere	Riduce la memoria usata e aumenta la sicurezza del layer di virtualizzazione
Lightweight OS	Sistemi operativi minimali e specializzati per l'esecuzione di applicazioni containerizzate.	Realizza un ambiente per il deployment dei container, veloce scalabile e con un basso consumo di memoria
Unikernel	Sistema operativo specializzato e progettato per eseguire una singola applicazione con un overhead minimo	Leggeri, veloci nel esecuzione e avvio, consumano poca memoria, riducono la superficie d'attacco e forniscono un forte isolamento. Sono però meno flessibili delle VM e container, più difficili da scalare anche a causa del supporto limitato e della difficile orchestrazione
WebAssembly	Formato di codice binario sicuro e portatile progettato per un'esecuzione efficiente e una rappresentazione compatta	Esegue in un ambiente sandbox e memory-safe, ha velocità di esecuzione quasi-nativa ed è indipendente dal linguaggio, piattaforma e dal HW. Svantaggi: non esiste un runtime standard e completo e non supporta nativamente la comunicazione tra due applicazioni wasm

Protocolli multicast	Descrizione	Ottimizzazioni e svantaggi
Flooding	il nodo richiedente P manda la query ai suoi vicini che la ignorano se l'hanno già ricevuta, rispondono se possiedono la risorsa o altrimenti inoltrano la query ai loro vicini, eccetto P. La risposta può essere mandata direttamente dal proprietario della risorsa al richiedente oppure la risposta segue lo stesso percorso dalla query attraverso il query ID senza dover ricalcolare il percorso per l'instradamento, riducendo il rischio di cicli e instradamento bilanciato.	Si può utilizzare un TTL, o assegnare un ID univoco alla query. Svantaggi:overhead di comunicazione anche le query senza successo consumano rete,nessuna garanzia che il proprietario della risorsa sia raggiunto, se i "vicini" sono fisicamente distante, si ha latenza elevata.
Random walk	il nodo richiedente P manda la query ad un suo vicino scelto casualmente che risponde se possiede la risorsa oppure inoltra la richiesta ad uno dei suoi vicini sempre scelto casualmente. Un'ottimizzazione è il k-random walk dove, il nodo richiedente inizia k random walk indipendenti, mandando la query a k vicini casuali bilanciando il traffico.	Ulteriori ottimizzazioni sono anche qui un TTL e un meccanismo per chiedere all'originator se serve ancora inoltrare la richiesta della risorsa o no. Svantaggi: si hanno meno messaggi e meno congestione ma i tempi di risposta sono più grandi
Anti-entropy	ogni nodo periodicamente seleziona un altro nodo casualmente, per scambiare informazioni con l'obiettivo di raggiungere uno stato identico su entrambi i nodi. In particolare, un nodo P seleziona un nodo Q casualmente e sceglie una politica tra push, pull e push-pull, la politica più efficiente, per scambiare informazioni.	
Rumor spreading	un nodo che ha un nuovo aggiornamento (è stato contaminato) periodicamente seleziona F>=1 nodi e invia il proprio aggiornamento. Tali F nodi possono poi scegliere se diffondere l'aggiornamento o meno. In particolare: un nodo P che ha un aggiornamento da comunicare contatta un nodo Q casuale e inoltra l'aggiornamento.	